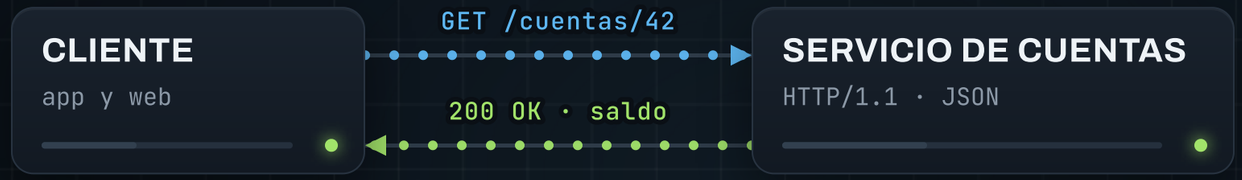
Recursos y verbos.

Lo que ya vimos en la S3.

REST nombra recursos (cuentas, pedidos) y los mueve con verbos.

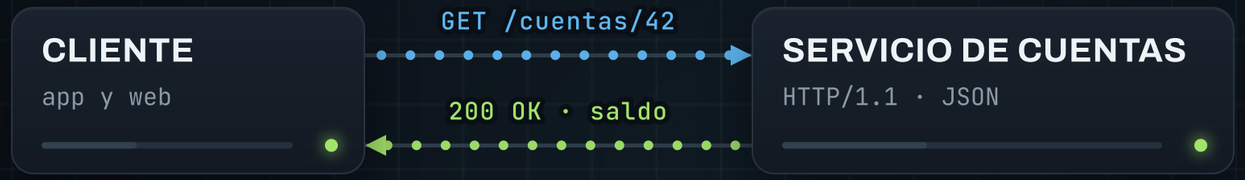
Recursos y verbos.

Lo que ya vimos en la S3.



Recursos y verbos.

Lo que ya vimos en la S3.



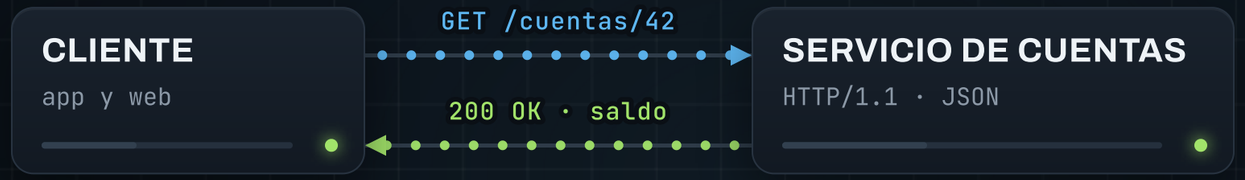
EL CÓDIGO ES CONTRATO

- **2xx** · funcionó
- **4xx** · tu error
- **5xx** · el nuestro

Dos equis funcionó, cuatro equis es tu error, cinco equis es el nuestro.

Recursos y verbos.

Lo que ya vimos en la S3.



REST

Recursos con nombre y verbos con contrato: **la red se vuelve explícita**.

S3 **MISMA LLAVE, MISMO EFECTO**

EL CÓDIGO ES CONTRATO

2xx · funcionó

4xx · tu error

5xx · el nuestro

Y lo de la S3: repetir la misma petición no debe cobrar dos veces.

Seis bases.

Y una de pilón.

1 · RECURSOS

/usuarios, /pedidos: sustantivos con dirección

Seis bases.

Y una de pilón.

1 · RECURSOS

/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES

casi siempre JSON, pero no es obligatorio

Seis bases.

Y una de pilón.

1 · RECURSOS

/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES

casi siempre JSON, pero no es obligatorio

3 · MÉTODOS PREDECIBLES

GET, POST, PUT, DELETE hacen lo que prometen

`POST /usuarios/42/borrar`

`DELETE /usuarios/42`

Tres: métodos predecibles. La acción va en el verbo, nunca en la URL.

Seis bases.

Y una de pilón.

1 · RECURSOS
/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES
casi siempre JSON, pero no es obligatorio

3 · MÉTODOS PREDECIBLES
GET, POST, PUT, DELETE hacen lo que prometen

4 · SIN ESTADO
cada petición trae lo necesario para entenderse sola

`POST /usuarios/42/borrar`

`DELETE /usuarios/42`

Cuatro: sin estado. Cada petición trae lo que necesita para entenderse sola.

Seis bases.

Y una de pilón.

1 · RECURSOS
/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES
casi siempre JSON, pero no es obligatorio

3 · MÉTODOS PREDECIBLES
GET, POST, PUT, DELETE hacen lo que prometen

4 · SIN ESTADO
cada petición trae lo necesario para entenderse sola

5 · INTERFAZ UNIFORME
las mismas reglas para todos los recursos

`POST /usuarios/42/borrar`

`DELETE /usuarios/42`

Cinco: interfaz uniforme. Las mismas reglas para todos los recursos.

Seis bases.

Y una de pilón.

1 · RECURSOS
/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES
casi siempre JSON, pero no es obligatorio

3 · MÉTODOS PREDECIBLES
GET, POST, PUT, DELETE hacen lo que prometen

4 · SIN ESTADO
cada petición trae lo necesario para entenderse sola

5 · INTERFAZ UNIFORME
las mismas reglas para todos los recursos

6 · CLIENTE Y SERVIDOR APARTE
el cliente no sabe cómo guardas los datos

`POST /usuarios/42/borrar`

`DELETE /usuarios/42`

Seis: cliente y servidor aparte. El cliente no sabe cómo guardas los datos.

Seis bases.

Y una de pilón.

1 · RECURSOS
/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES
casi siempre JSON, pero no es obligatorio

3 · MÉTODOS PREDECIBLES
GET, POST, PUT, DELETE hacen lo que prometen

4 · SIN ESTADO
cada petición trae lo necesario para entenderse sola

5 · INTERFAZ UNIFORME
las mismas reglas para todos los recursos

6 · CLIENTE Y SERVIDOR APARTE
el cliente no sabe cómo guardas los datos

`POST /usuarios/42/borrar`

`DELETE /usuarios/42`

EXTRA: RESPUESTAS CACHEABLES

Y de pilón: respuestas cacheables. Quien va en medio puede guardar la copia.

Seis bases.

Y una de pilón.

1 · RECURSOS
/usuarios, /pedidos: sustantivos con dirección

2 · REPRESENTACIONES
casi siempre JSON, pero no es obligatorio

3 · MÉTODOS PREDECIBLES
GET, POST, PUT, DELETE hacen lo que prometen

4 · SIN ESTADO
cada petición trae lo necesario para entenderse sola

5 · INTERFAZ UNIFORME
las mismas reglas para todos los recursos

6 · CLIENTE Y SERVIDOR APARTE
el cliente no sabe cómo guardas los datos

PARA DISCUTIR

¿Por qué REST funciona bien en sistemas distribuidos, **más allá** de GET, POST, PUT y DELETE?

`POST /usuarios/42/borrar`

`DELETE /usuarios/42`

EXTRA: RESPUESTAS CACHEABLES

Ahora la pregunta del grupo: ¿por qué esto **aguanta** un sistema distribuido?

El contrato opcional.

Lo que igual hay que resolver.

LO QUE EL CLIENTE NECESITA

Paginación diez mil filas no caben en una respuesta

Versionado /v1 y /v2 conviven mientras migran

Rate limit 429 y Retry-After: cuántas por minuto

REST se ve simple, pero el cliente pide más: paginar, versionar, aguantar límites.

El contrato opcional.

Lo que igual hay que resolver.

LO QUE EL CLIENTE NECESITA

Paginación diez mil filas no caben en una respuesta

Versionado /v1 y /v2 conviven mientras migran

Rate limit 429 y Retry-After: cuántas por minuto

LO QUE LA RED OBLIGA

Timeouts soltar el hilo antes de morir esperando

Reintentos espera exponencial con jitter

Idempotencia la misma llave, el mismo efecto

Y la red obliga al resto: timeouts, reintentos con espera y idempotencia.

El contrato opcional.

Lo que igual hay que resolver.

LO QUE EL CLIENTE NECESITA

- **Paginación** diez mil filas no caben en una respuesta
- **Versionado** /v1 y /v2 conviven mientras migran
- **Rate limit** 429 y Retry-After: cuántas por minuto

LO QUE LA RED OBLIGA

- **Timeouts** soltar el hilo antes de morir esperando
- **Reintentos** espera exponencial con jitter
- **Idempotencia** la misma llave, el mismo efecto

OPENAPI: EL CONTRATO ES OPCIONAL

El contrato existe, pero es opcional: se escribe aparte… o no se escribe.

El contrato opcional.

Lo que igual hay que resolver.

LO QUE EL CLIENTE NECESITA

Paginación diez mil filas no caben en una respuesta

Versionado /v1 y /v2 conviven mientras migran

Rate limit 429 y Retry-After: cuántas por minuto

LO QUE LA RED OBLIGA

Timeouts soltar el hilo antes de morir esperando

Reintentos espera exponencial con jitter

Idempotencia la misma llave, el mismo efecto

LA LETRA CHICA

Nada de esto lo impone el estilo: **se acuerda**, se documenta y se vuelve a acordar.

OPENAPI: EL CONTRATO ES OPCIONAL

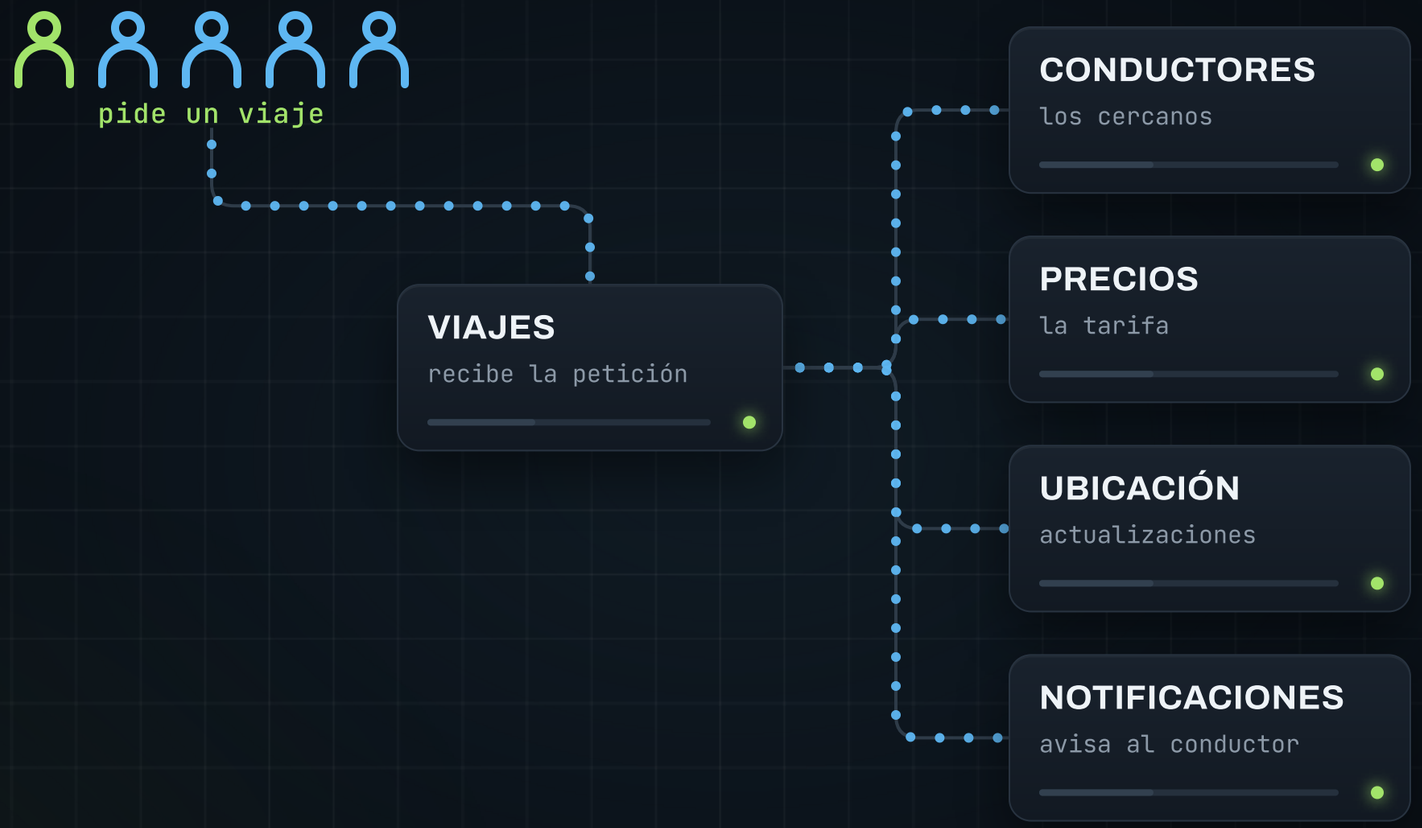
Una acción.

Muchas llamadas.

Un usuario pide un viaje. Para él es una acción.

Una acción.

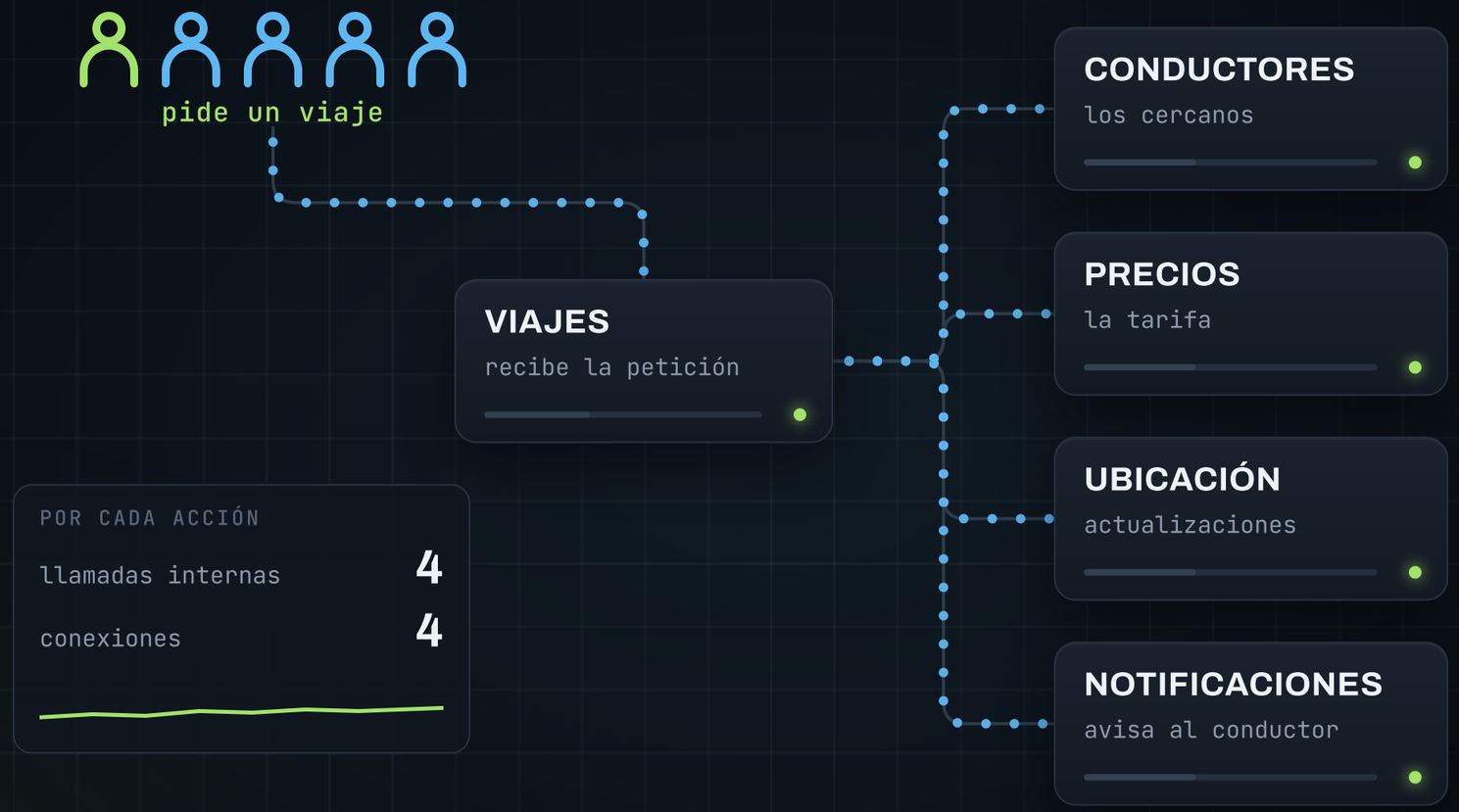
Muchas llamadas.



Adentro, viajes llama a conductores, precios, ubicación y notificaciones.

Una acción.

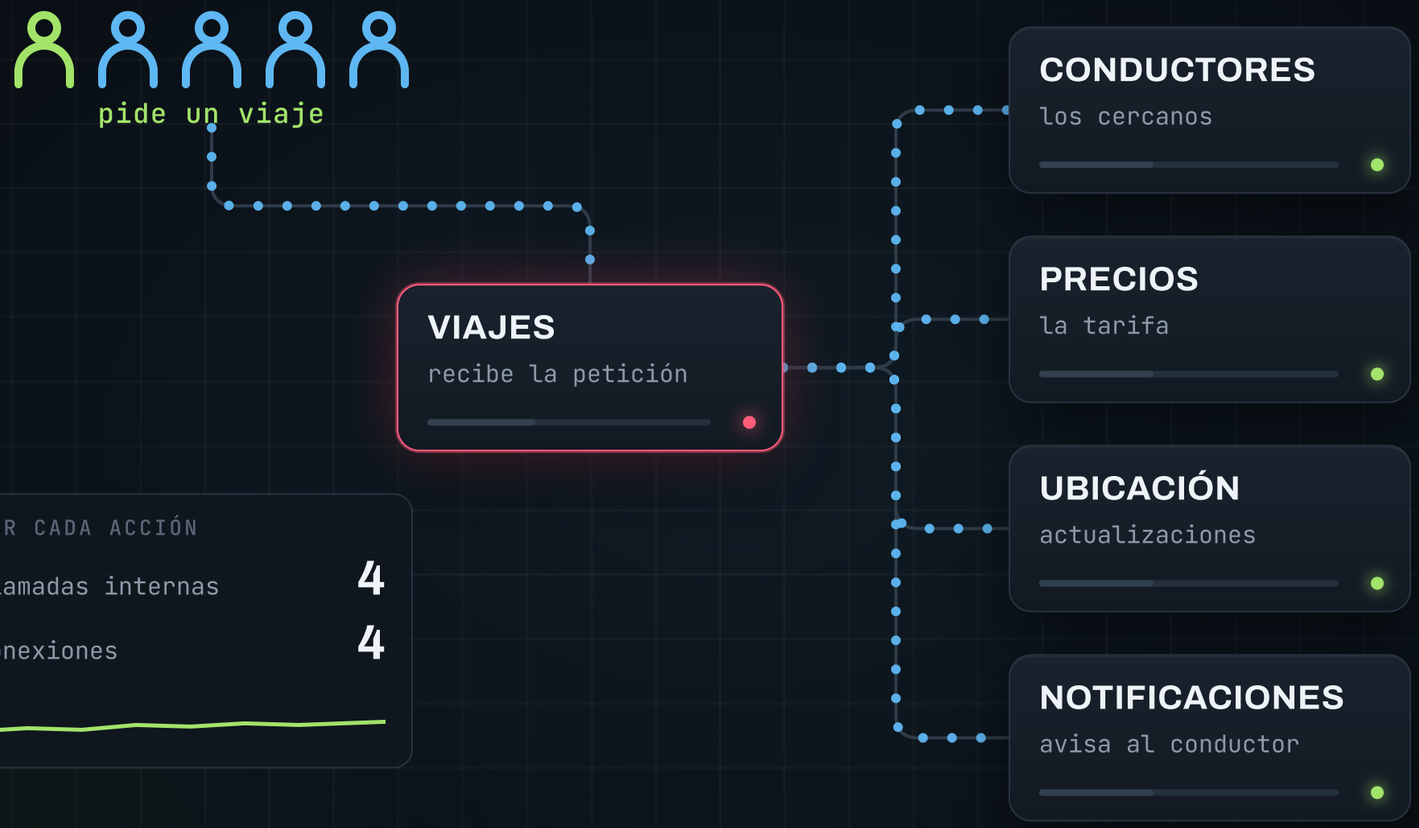
Muchas llamadas.



Cuatro llamadas por una. Y en hora pico, miles por segundo.

Una acción.

Muchas llamadas.



POR CADA ACCIÓN

llamadas internas	4
conexiones	4

AMPLIFICACIÓN

Una petición del usuario se convierte en **N llamadas** entre servicios.

Una conexión.

Muchos streams.

SIN HTTP/2

CONEXIÓN 1 · TCP + TLS

CONEXIÓN 2 · TCP + TLS

CONEXIÓN 3 · TCP + TLS

Antes, cada llamada abría su propia conexión: TCP, TLS y a empezar de nuevo.

Una conexión.

Muchos streams.

SIN HTTP/2

CONEXIÓN 1 · TCP + TLS

CONEXIÓN 2 · TCP + TLS

CONEXIÓN 3 · TCP + TLS

O UNA SOLA, Y HACEN FILA

Y si compartían una, las peticiones hacían fila, una detrás de otra.

Una conexión.

Muchos streams.

SIN HTTP/2

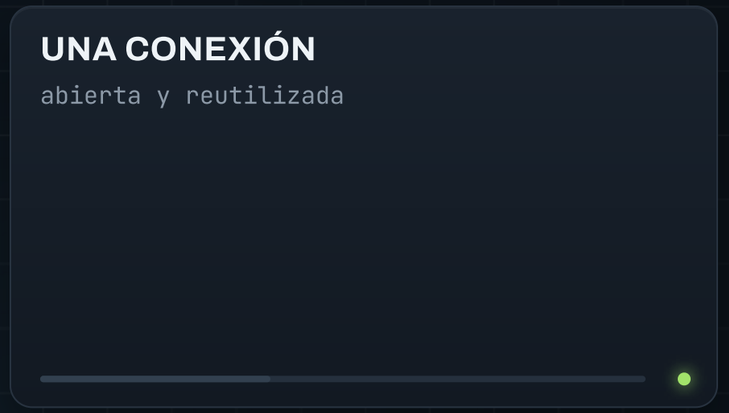
CONEXIÓN 1 · TCP + TLS

CONEXIÓN 2 · TCP + TLS

CONEXIÓN 3 · TCP + TLS

O UNA SOLA, Y HACEN FILA

CON HTTP/2



HTTP/2 deja una conexión abierta entre los dos servicios.

Una conexión.

Muchos streams.

SIN HTTP/2

CONEXIÓN 1 · TCP + TLS

CONEXIÓN 2 · TCP + TLS

CONEXIÓN 3 · TCP + TLS

O UNA SOLA, Y HACEN FILA

CON HTTP/2

Adentro, cada petición viaja en su propio stream, al mismo tiempo.

Una conexión.

Muchos streams.

HTTP/2

Una conexión con varios **streams** a la vez, y encabezados que no se repiten enteros.

SIN HTTP/2

CONEXIÓN 1 · TCP + TLS

CONEXIÓN 2 · TCP + TLS

CONEXIÓN 3 · TCP + TLS

O UNA SOLA, Y HACEN FILA

CON HTTP/2

UNA CONEXIÓN

abierta y reutilizada

STREAM 1

STREAM 2

STREAM 3

ENCABEZADOS COMPRIMIDOS

Los mismos datos.

Menos bytes.

```
{"id":42,"nombre":"Ada","activo":true}
```

JSON · EL NOMBRE DE CADA CAMPO VIAJA, SIEMPRE

EL MISMO DATO EN EL CABLE

REST + JSON	180 B
gRPC + Protobuf	180 B

JSON es texto y se lee de corrido, pero carga el nombre de cada campo.

Los mismos datos.

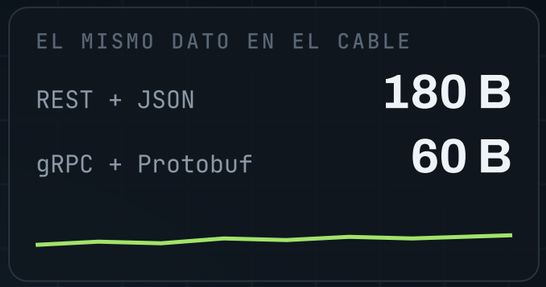
Menos bytes.

```
{"id":42,"nombre":"Ada","activo":true}
```

JSON · EL NOMBRE DE CADA CAMPO VIAJA, SIEMPRE

```
08 2A 12 03 41 64 61 18 01
```

PROTOBUF · NÚMERO DE CAMPO Y VALOR



Protobuf es binario: número de campo y valor.
Sesenta bytes en vez de ciento ochenta.

Los mismos datos.

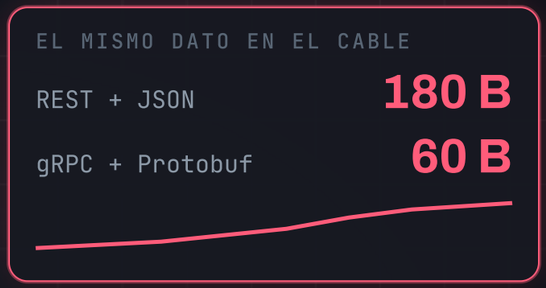
Menos bytes.

```
{"id":42,"nombre":"Ada","activo":true}
```

JSON · EL NOMBRE DE CADA CAMPO VIAJA, SIEMPRE

```
08 2A 12 03 41 64 61 18 01
```

PROTOBUF · NÚMERO DE CAMPO Y VALOR



Una llamada no se nota. Mil por segundo, sí.

Los mismos datos.

Menos bytes.

```
{"id":42,"nombre":"Ada","activo":true}
```

JSON · EL NOMBRE DE CADA CAMPO VIAJA, SIEMPRE

```
08 2A 12 03 41 64 61 18 01
```

PROTOBUF · NÚMERO DE CAMPO Y VALOR

EL MISMO DATO EN EL CABLE

REST + JSON	180 B
gRPC + Protobuf	60 B

EL NOMBRE LO PONE EL .PROTO

POR QUÉ PESA MENOS

Si los dos lados ya tienen el contrato, **los nombres no viajan**.

No se perdió nada: el nombre lo pone el contrato, y el contrato ya lo tienen los dos.

Un archivo.

Y el código se genera.

CONDUCTORES.PROTO

```
service Conductores {
rpc BuscarCercanos(Ubicacion)
returns (Conductores);
}
message Ubicacion {
double lat = 1;
double lon = 2;
}
```

Los dos lados acuerdan un archivo: servicios, métodos y mensajes.

Un archivo.

Y el código se genera.

CONDUCTORES.PROTO

```
service Conductores {
rpc BuscarCercanos(Ubicacion)
returns (Conductores);
}
message Ubicacion {
double lat = 1;
double lon = 2;
}
```

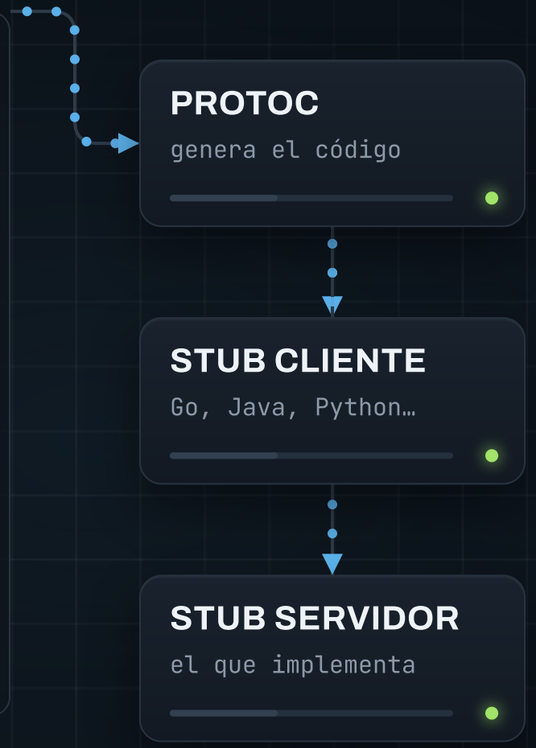
De ahí, protoc genera el código de cliente y servidor.

Un archivo.

Y el código se genera.

CONDUCTORES.PROTO

```
service Conductores {
rpc BuscarCercanos(Ubicacion)
returns (Conductores);
}
message Ubicacion {
double lat = 1;
double lon = 2;
}
```



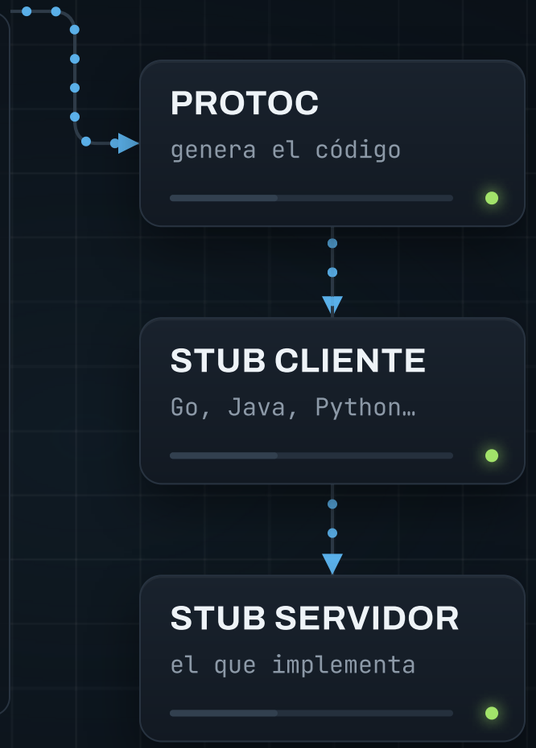
Cada quien en su lenguaje: Go llamando a Python, Java llamando a Node.

Un archivo.

Y el código se genera.

CONDUCTORES.PROTO

```
service Conductores {
rpc BuscarCercanos(Ubicacion)
returns (Conductores);
}
message Ubicacion {
double lat = 1;
double lon = 2;
}
```




```
conductores.buscarCercanos(ubicacion)
```

Y en el código se ve así: una función normal.

Un archivo.

Y el código se genera.

CONDUCTORES.PROTO

```
service Conductores {
rpc BuscarCercanos(Ubicacion)
returns (Conductores);
}
message Ubicacion {
double lat = 1;
double lon = 2;
}
```

RPC

Llamas un método que vive en otro servidor **como si fuera local**.

```
conductores.buscarCercanos(ubicacion)
```

Debajo, gRPC serializa, manda, espera y devuelve. Eso es remote procedure call.

No siempre.

Es pregunta y respuesta.

UNARIO

una petición → una respuesta

El unario es el clásico: una petición, una respuesta.

No siempre.

Es pregunta y respuesta.

UNARIO
una petición → una respuesta

STREAMING DEL SERVIDOR
una petición → muchas respuestas

Con streaming del servidor, pides una vez y te van llegando respuestas.

No siempre.

Es pregunta y respuesta.

UNARIO
una petición → una respuesta

STREAMING DEL SERVIDOR
una petición → muchas respuestas

STREAMING DEL CLIENTE
muchas peticiones → una respuesta

Con streaming del cliente, tú mandas muchas y él contesta al final.

No siempre.

Es pregunta y respuesta.

UNARIO
una petición → una respuesta

STREAMING DEL SERVIDOR
una petición → muchas respuestas

STREAMING DEL CLIENTE
muchas peticiones → una respuesta

BIDIRECCIONAL
los dos hablan a la vez

Y bidireccional: los dos hablan al mismo tiempo, por el mismo stream.

No siempre.

Es pregunta y respuesta.

UNARIO
una petición → una respuesta

STREAMING DEL SERVIDOR
una petición → muchas respuestas

STREAMING DEL CLIENTE
muchas peticiones → una respuesta

BIDIRECCIONAL
los dos hablan a la vez

CUATRO MODOS

HTTP/2 abre streams, así que la llamada deja de ser **solo pregunta y respuesta**.

PARA QUÉ SIRVE CADA UNO

Unario el noventa por ciento de las llamadas

Del servidor la posición del conductor, cada segundo

Del cliente subir un archivo por partes

Bidireccional chat, telemetría, juegos

Esto es lo que HTTP/1.1 no daba, y por eso venía con REST limitado.

REST o gRPC.

Mismo problema, precios distintos.

	REST	gRPC
Formato en el cable	JSON, texto	Protobuf, binario
Contrato	opcional (OpenAPI)	obligatorio (.proto)
Transporte	HTTP/1.1 o superior	HTTP/2
Streaming	limitado	nativo, bidireccional
Depuración	curl y navegador	grpcurl
Clientes	cualquiera con HTTP	código generado
Uso dominante	el borde, APIs públicas	interno, servicio a servicio

Las mismas siete filas, leídas como precios y no como méritos.

REST o gRPC.

Mismo problema, precios distintos.

	REST	gRPC
Formato en el cable	JSON, texto	Protobuf, binario
Contrato	opcional (OpenAPI)	obligatorio (.proto)
Transporte	HTTP/1.1 o superior	HTTP/2
Streaming	limitado	nativo, bidireccional
Depuración	curl y navegador	grpcurl
Clientes	cualquiera con HTTP	código generado
Uso dominante	el borde, APIs públicas	interno, servicio a servicio

La frontera pública favorece REST: cualquier cliente lo lee y se depura con curl.

REST o gRPC.

Mismo problema, precios distintos.

	REST	gRPC
Formato en el cable	JSON, texto	Protobuf, binario
Contrato	opcional (OpenAPI)	obligatorio (.proto)
Transporte	HTTP/1.1 o superior	HTTP/2
Streaming	limitado	nativo, bidireccional
Depuración	curl y navegador	grpcurl
Clientes	cualquiera con HTTP	código generado
Uso dominante	el borde, APIs públicas	interno, servicio a servicio

El tráfico interno favorece gRPC: miles de llamadas, y el contrato evita divergencias.

REST o gRPC.

Mismo problema, precios distintos.

	REST	gRPC
Formato en el cable	JSON, texto	Protobuf, binario
Contrato	opcional (OpenAPI)	obligatorio (.proto)
Transporte	HTTP/1.1 o superior	HTTP/2
Streaming	limitado	nativo, bidireccional
Depuración	curl y navegador	grpcurl
Clientes	cualquiera con HTTP	código generado
Uso dominante	el borde, APIs públicas	interno, servicio a servicio

EL NAVEGADOR NO HABLA GRPC: GRPC-WEB

Y hay un límite duro: el navegador no habla gRPC nativo; gRPC-Web tiende el puente.

REST o gRPC.

Mismo problema, precios distintos.

	REST	gRPC
Formato en el cable	JSON, texto	Protobuf, binario
Contrato	opcional (OpenAPI)	obligatorio (.proto)
Transporte	HTTP/1.1 o superior	HTTP/2
Streaming	limitado	nativo, bidireccional
Depuración	curl y navegador	grpcurl
Clientes	cualquiera con HTTP	código generado
Uso dominante	el borde, APIs públicas	interno, servicio a servicio

NO ES UNA REGLA

gRPC **no reemplaza** a HTTP: corre encima. Es un trade-off, no un ganador.

EL NAVEGADOR NO HABLA GRPC: GRPC-WEB

MEXI BANCO: GATEWAY → SERVICIOS, REST · TRANSFERENCIAS → CUENTAS, CANDIDATO A GRPC

En nuestro sistema: el gateway habla REST; transferencias a cuentas es el candidato.

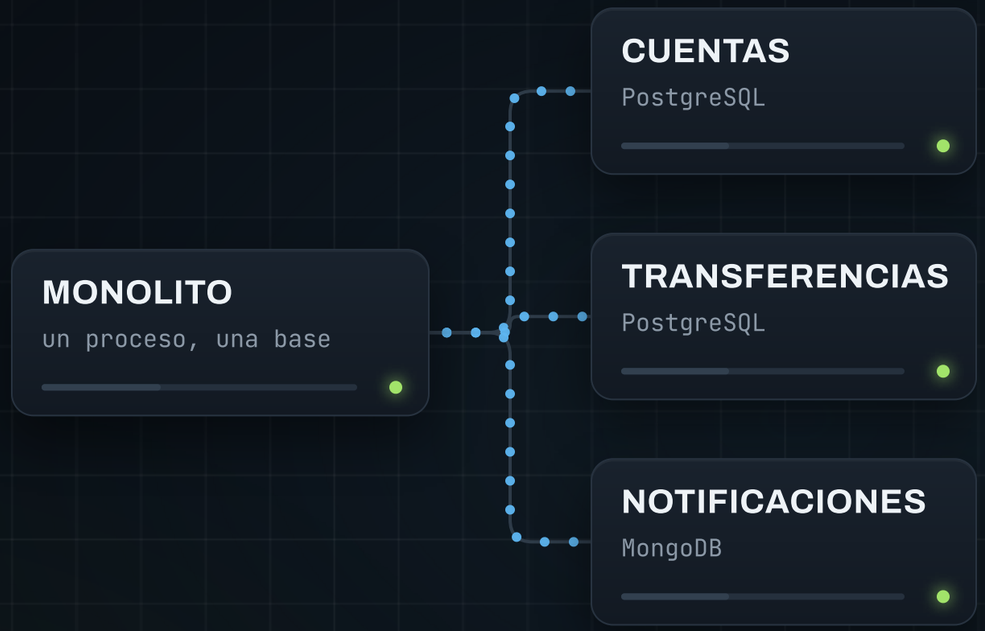
Partir el monolito.

Crea seis preguntas.

Ayer era un proceso: las llamadas eran funciones en memoria.

Partir el monolito.

Crea seis preguntas.



Hoy son tres servicios, cada uno dueño de su base.

Partir el monolito.

Crea seis preguntas.

Y cada flecha entre ellos ya cruza la red: puede tardar, fallar o repetirse.

Partir el monolito.

Crea seis preguntas.

Eso abre seis preguntas que el monolito nunca tuvo que contestar.

Partir el monolito.

Crea seis preguntas.

EL COSTO DE PARTIR

Lo que era una llamada en memoria ahora **cruza la red**, con todo lo que eso trae.

Cada pieza de hoy existe porque responde una de esas seis.

Entrar, encontrar.

Y saber con qué valores.

Sin gateway, cada servicio expone su puerto y el cliente se acopla a la topología.

Entrar, encontrar.

Y saber con qué valores.

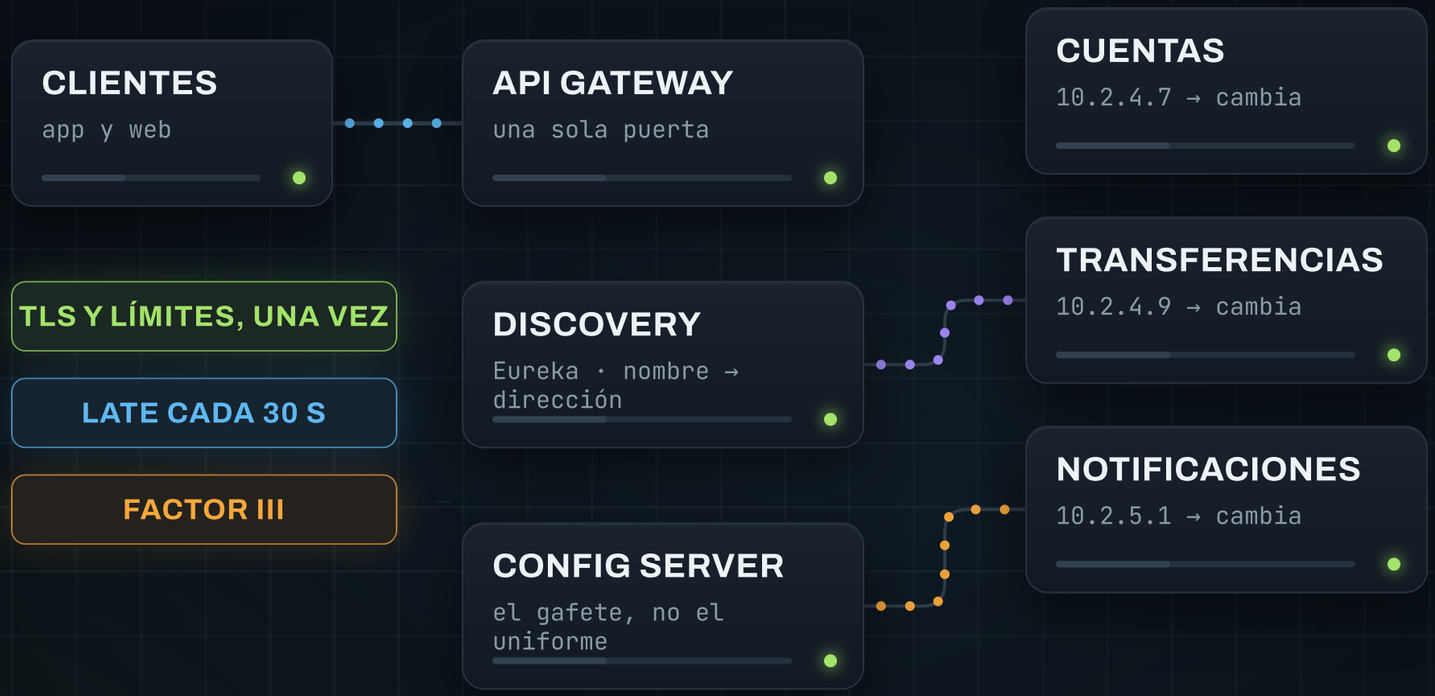
Entrar, encontrar.

Y saber con qué valores.

En contenedores la dirección cambia en cada arranque: el nombre se separa de la IP.

Entrar, encontrar.

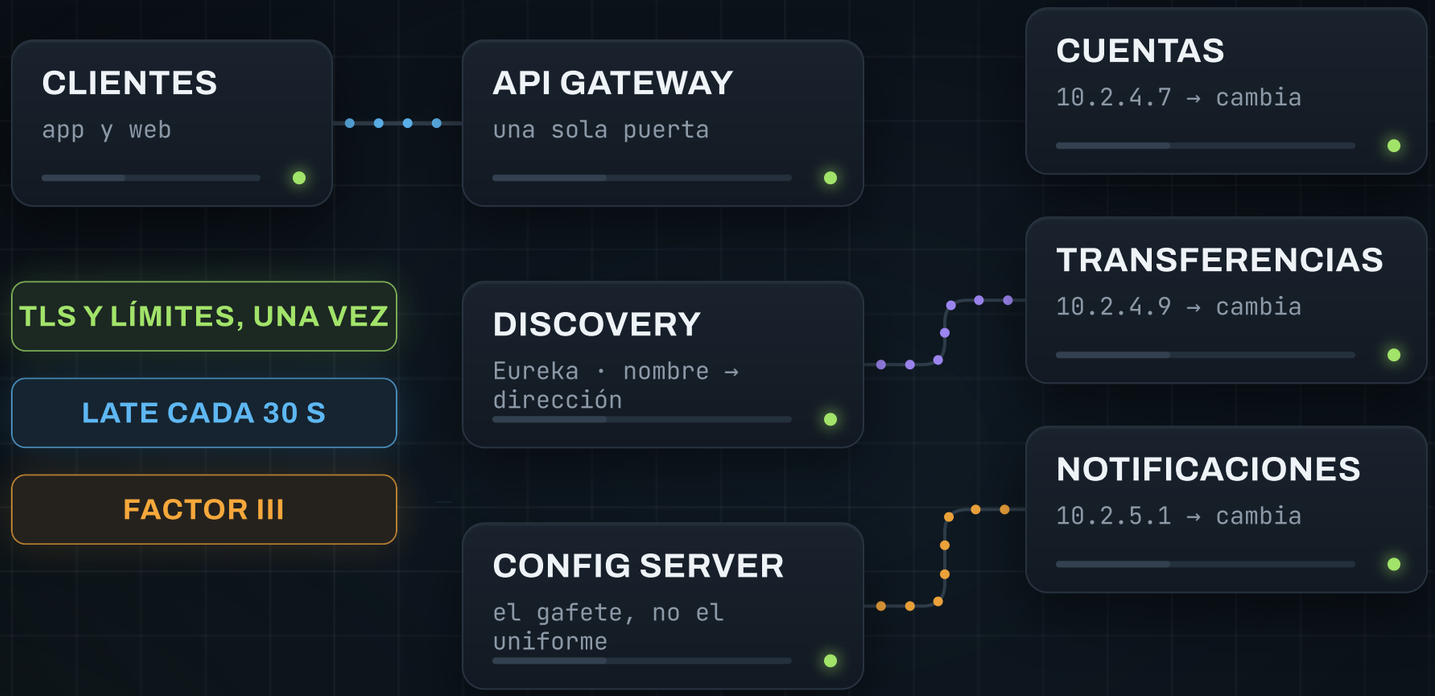
Y saber con qué valores.



Y la misma imagen corre en dev y en prod: la configuración va por fuera.

Entrar, encontrar.

Y saber con qué valores.



LAS PIEZAS DE ENTRADA

Quién entra, a quién llama y **con qué valores** corre cada instancia.

Tres piezas, tres preguntas: por dónde entro, a quién llamo, con qué valores corro.

Aguantar, no repetir.

Y poder contar la historia.

Llamada directa: si notificaciones está caído, la transferencia falla con él.

Aguantar, no repetir.

Y poder contar la historia.

El broker guarda el mensaje: entrega al menos una vez, cuando el otro vuelva.

Aguantar, no repetir.

Y poder contar la historia.

La base es el recurso caro y compartido, y las mismas lecturas se repiten mil veces.

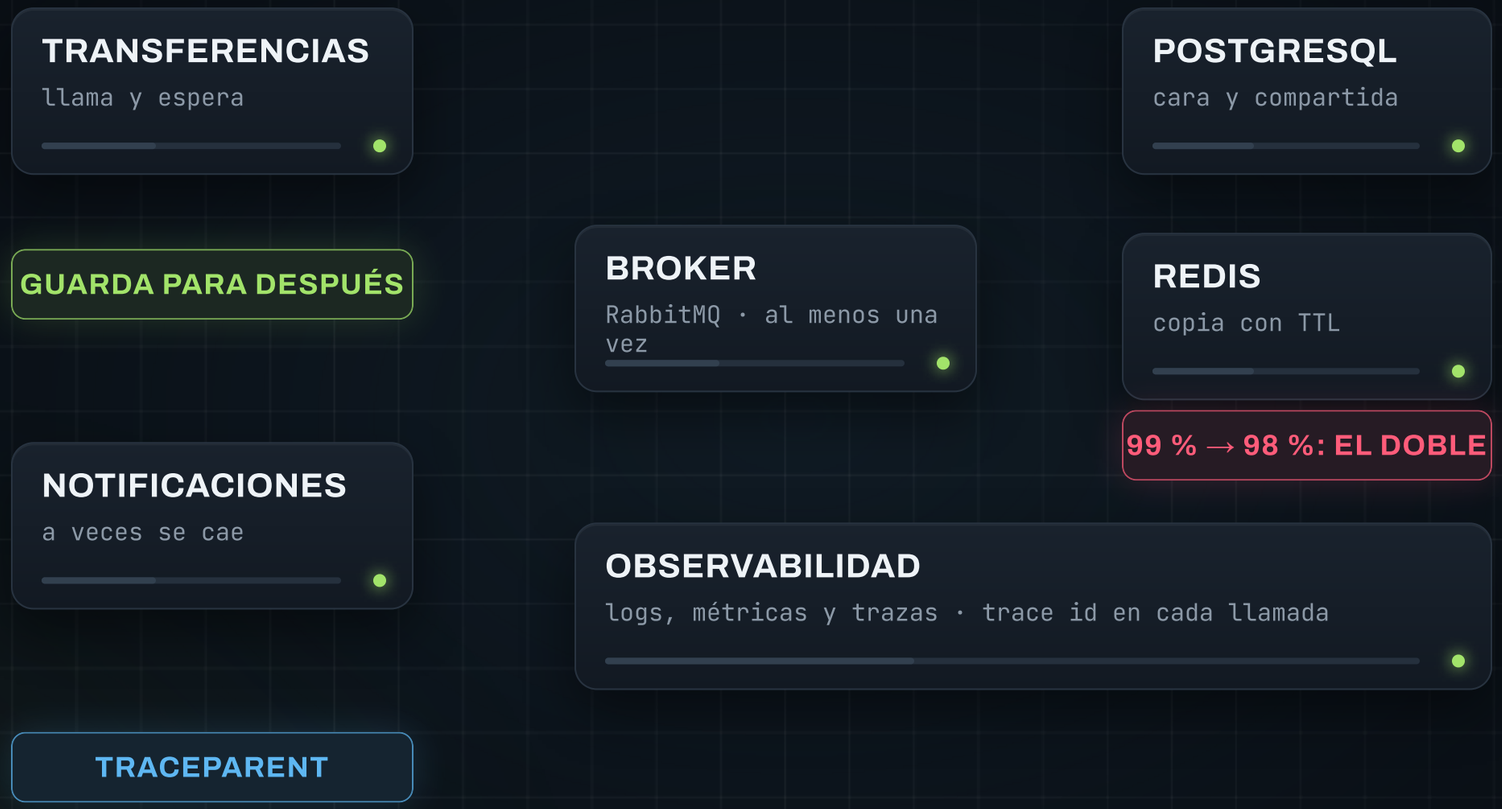
Aguantar, no repetir.

Y poder contar la historia.

La caché es una copia que se permite estar vieja: si está, hit; si no, miss y TTL.

Aguantar, no repetir.

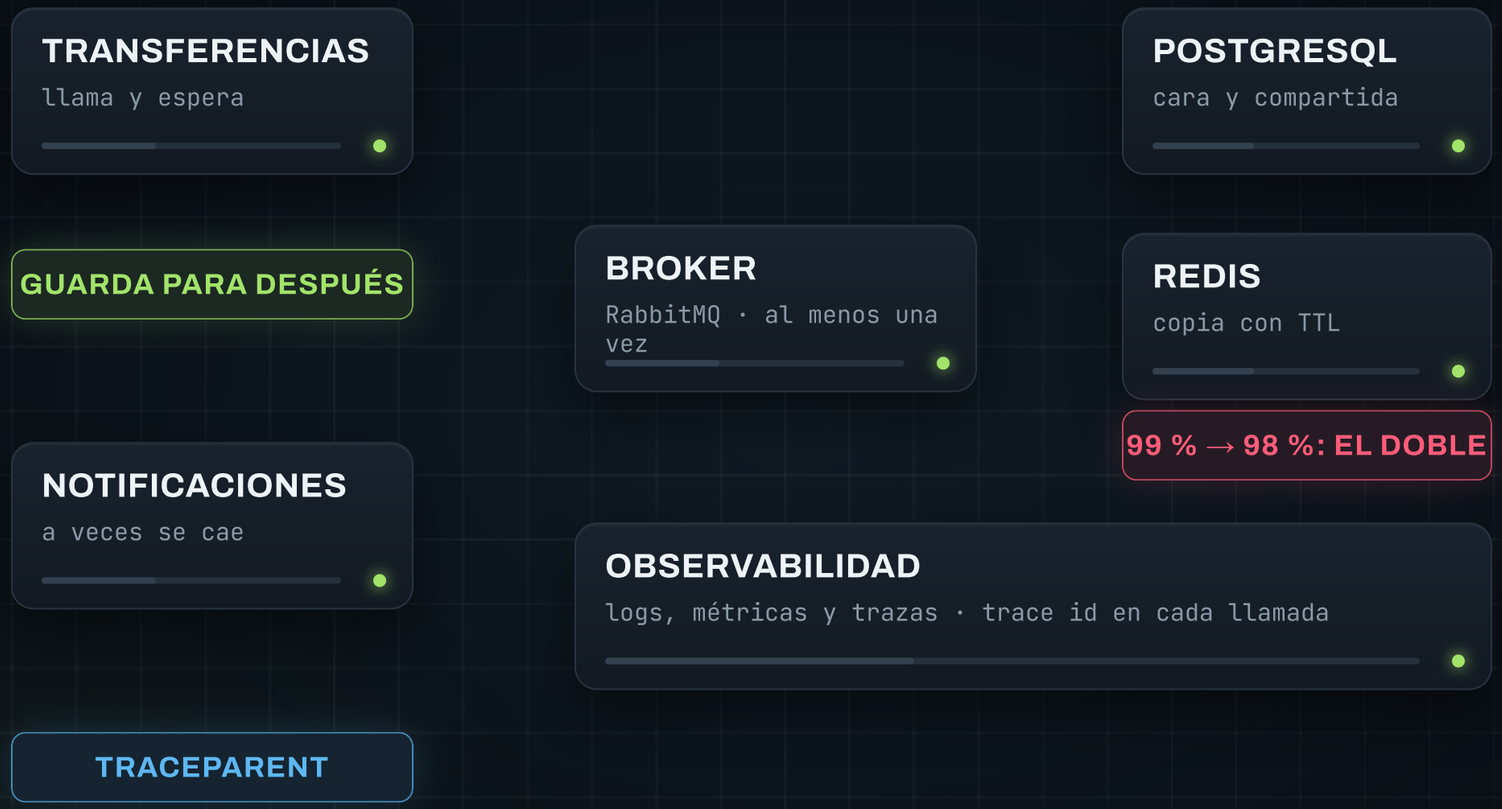
Y poder contar la historia.



Y cuando la operación cruza cuatro servicios, sin trace id hay veinte bitácoras y ninguna historia.

Aguantar, no repetir.

Y poder contar la historia.



LAS PIEZAS DE RESILIENCIA

Desacoplar la disponibilidad, no repetir trabajo caro y **poder reconstruir** una operación.

Tres piezas más: aguantar que el otro falle, no repetir trabajo y ver el recorrido.

Cuatro factores.

Ya los vieron hoy.

III · CONFIG

`config server; en el lab, el .env`

La configuración fuera de la imagen: eso es el factor tres.

Cuatro factores.

Ya los vieron hoy.

III · CONFIG
`config server; en el lab, el .env`

VI · PROCESOS SIN ESTADO
`discovery y N réplicas iguales`

Cualquier instancia atiende porque el estado no vive en ella: factor seis.

Cuatro factores.

Ya los vieron hoy.

III · CONFIG
config server; en el lab, el .env

VI · PROCESOS SIN ESTADO
discovery y N réplicas iguales

IV · BACKING SERVICES
broker, caché y base, por URL

Broker, caché y base son adjuntos por URL: factor cuatro.

Cuatro factores.

Ya los vieron hoy.

III · CONFIG
config server; en el lab, el .env

VI · PROCESOS SIN ESTADO
discovery y N réplicas iguales

IV · BACKING SERVICES
broker, caché y base, por URL

XI · LOGS
flujos a stdout; otro los junta

Y los logs son flujos que alguien más junta: factor once.

Cuatro factores.

Ya los vieron hoy.

III · CONFIG
`config server; en el lab, el .env`

VI · PROCESOS SIN ESTADO
`discovery y N réplicas iguales`

IV · BACKING SERVICES
`broker, caché y base, por URL`

XI · LOGS
`flujos a stdout; otro los junta`

LAS SEIS PIEZAS, UNA LÍNEA CADA UNA

Gateway una puerta

Discovery nombre estable

Config fuera de la imagen

Broker al menos una vez

Caché copia con TTL

Observabilidad trace id

Seis piezas, seis preguntas, y cuatro factores que ya quedaron ejercidos.

Cuatro factores.

Ya los vieron hoy.

III · CONFIG
config server; en el lab, el .env

VI · PROCESOS SIN ESTADO
discovery y N réplicas iguales

IV · BACKING SERVICES
broker, caché y base, por URL

XI · LOGS
flujos a stdout; otro los junta

LA REGLA

Ninguna pieza se agrega por moda: cada una **paga una pregunta** que abrió partir el monolito.

LAS SEIS PIEZAS, UNA LÍNEA CADA UNA

Gateway una puerta

Discovery nombre estable

Config fuera de la imagen

Broker al menos una vez

Caché copia con TTL

Observabilidad trace id

Lo que sigue el sábado: el sistema corriendo, y la decisión de FarmaYa.